MANCHESTER 1824
The University of Manchester

Following Topological Defect Annihilation with Machine Learning Algorithms

Katherine Skingle

In Collaboration with Laura Hernández Martín

Department of Physics and Astronomy
The University of Manchester

MPhys Project Report

The aim of this project was to use nanoparticle-doped liquid crystal solutions and investigate the impact of the nanoparticles on universal scaling laws of topological defect annihilation, by using machine learning to track the defects and thus investigate the dynamics of the system. The distance between annihilating defects is typically proportional to the square root of the time to annihilation. This relationship was tested for differently sized barium titanate nanoparticles at different concentrations, using quasi-2D nematic cells. Graphical fit results suggest that values of scaling exponent decay exponentially with nanoparticle size and concentration. This may therefore indicate a breakdown of scaling laws in other systems with topological defects in the presence of analogous objects.

1 Liquid Crystals

1.1 Introduction

1.1.1 Overview of Liquid Crystals

A liquid crystal is, as the name suggests, a liquid with some of the properties of a crystal, arising due to molecular order [1]. Typically, this means that the molecules in a liquid crystal point in a particular direction (orientational order) and may be fixed with respect to one another within a lattice (positional order); however, they still exhibit liquid properties, such as diffusion [1]. Changes in orientational order in regions of liquid crystal materials can lead to formation of defects [2], which create fascinating and diverse patterns within liquid crystal materials [3].

Liquid crystal molecules can be manipulated using electric fields [2], allowing them to have important applications in technology. For example, liquid crystals have been utilised to make Liquid Crystal Displays (LCDs), which are used in a variety of electronic goods from televisions to calculators [1, 4]. To control the amount of light emitted, an electric field manipulates a liquid crystal layer between two crossed polarisers to allow transmission of varying amounts of light in a display screen [5]. With low power consumption and small size, LCDs were revolutionary in the technology industry, and compare well with alternative screens based on price of manufacture, durability, brightness and resolution density [1, 5].

As well as technological applications, phenomena studied in liquid crystals have important scientific implications. The dynamics of liquid crystals, and of the topological defects within them, exist at distance and time scales which allow them to be studied experimentally with relative ease [6]. As spontaneous symmetry breaking results in creation of topological defects, they can be found in a wide variety of fields, from particle physics to cosmology [7]. For example, scaling laws relating to defect annihilation in liquid crystals may be analogous to defect annihilation in the early universe [2, 8]. Studying topological defects in liquid crystals may thus reveal information concerning early-universe phase transitions.

1.1.2 History of Liquid Crystals

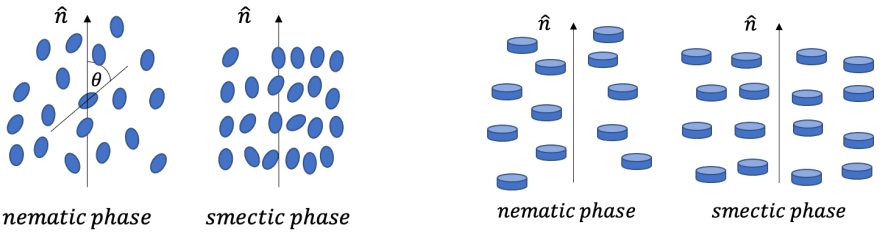
One of the oldest known liquid crystals is myelin, described by Rudolf Virchow in 1854 [9]. The first description of liquid crystals and associated phenomena, however, is typically considered to have been made when in 1888 Friedrich Reinitzer, a botanist by trade, observed a strange substance in the form of a derivative of cholesterol made from an extraction of a carrot root [1, 4]. It appeared to have two melting points [9]: at 145.5°C, it melted into a cloudy liquid from its solid state, and then at 178.5°C it clarified, while remaining in the liquid state. Upon cooling, it became cloudy again [10]. As crystallisation proceeded, the shade turned from a pale blue to a more vibrant blue-purple [1]. Reinitzer was aware of the work of Otto Lehmann, who studied crystallisation, and Reinitzer therefore sent Lehmann samples of the substance in a bid to better understand what was being observed [9]. Taking

note of its unusual behaviour, and the behaviour of some of his own samples, Lehmann began to suspect that he was observing a unique phase of matter, which he referred to using various descriptors: as soft crystals, later as crystalline fluids, and finally as liquid crystals [1]. Lehman and Reinitzer observed thread-like structures in these substances and proposed correctly, yet controversially, that certain liquid crystals contain rod-like molecules oriented along the long axis [3, 4].

By 1971, scientists discovered that liquid crystals could be ordered positionally as well as orientationally [3]. The tantalising applications that may have become available from this fascinating phase of matter, such as in LCDs, helped to ensure that research made leaps and bounds within the latter half of the 20th Century [1]. In the modern day, a huge variety of liquid crystal compounds with specific properties can be designed and synthesised out of specific molecular structures to best fit a range of applications [11].

1.2 Properties of Liquid Crystals

Liquid crystals are substances within a distinct stable phase of matter [2]. Thermodynamically speaking, they lie somewhere between isotropic liquids and solid crystals [12]. The self-ordering in liquid crystals is sometimes positional and always orientational [1]. The simplest type of liquid crystal is the nematic phase, from the Greek word for thread-like *nematos* [13], whereupon for rod-like or 'calamitic' molecules the molecules have a preferred orientation aligned along the long molecular axis [14]. However, they have no order with respect to position, making them the liquid crystal phase with the highest symmetry and the lowest degree of order [15]. Liquid crystals in the nematic phase therefore have high fluidity and low viscosity, making them suitable for liquid crystal displays; this will be discussed further in Section 1.4 [1]. Adding positional order to the liquid crystal denotes that it is in the smectic phase [16]. Illustrations of the nematic and smectic phases for calamitic liquid crystals are shown in Figure 1a. Alternatively, for disc-like (i.e. discotic) molecules, the nematic phase is characterised by the short axes orienting preferentially along the director, and the smectic phase by the additional tendency for the discs to be ordered within columns [1], shown in Figure 1b.



(a) Calamitic liquid crystal phases.

(b) Discotic liquid crystal phases.

FIG. 1. An illustration of calamitic and discotic liquid crystals within the nematic and smectic phases. The direction of orientation aligned with the long molecular axis, n, and the angle θ describe the orientational ordering [1].

Also shown in Figure 1b, the director, defined by the unit vector $\hat{n}$, denotes the preferential direction of the long molecular axis [1]. Deviation of the director described by the angle θ results in increased elastic energy [6]. The variation in orientational order described by θ is typically small at the microscopic level. For a 2D nematic system where

$$\hat{n} = (cos\theta, sin\theta), \tag{1}$$

using the approximation of a single elastic constant K in both directions, the free energy density w is given as

$$w = \frac{K}{2}((\frac{\partial\theta}{\partial x})^2 + (\frac{\partial\theta}{\partial y})^2). \tag{2}$$

The minimum of the free energy density in the 2D system is therefore given as the solution of Laplace's equation where $\Delta\theta$ is equal to zero [6].

All liquid crystals are anisotropic due to the ordering within the fluid, meaning properties exhibited within the liquid crystal are different in different directions [7]. However, the centres of mass of the molecules may have isotropic distribution [12]. In the case of the nematic and smectic phases, the liquid crystal is said to be uniaxial, as the one molecular axis of preferential orientation is different to the other two equivalent axes. Liquid crystal materials can also be biaxial, with three different molecular axes [1].

1.3 Topological Defects

Defects play a large role in determining the properties of a liquid crystal, from optical properties to electrical conductivity [2]. They can be points, lines (disinclinations), or sheets, arising due to variations in positional or orientational order [1] where the director can no longer be uniquely defined within a certain region [3]. The assumption that the orientational order is continuous in a liquid crystal, i.e. that θ is negligibly small, fails in a region which contains a topological defect [1]. An illustration of possible types of distortion of molecular order can be seen in Figure 2. Defects are typically induced within liquid crystal materials via pressure or temperature changes, or in the presence of an electric field, due to change in the free energy of the liquid crystal. The elastic constants of defects are orders of magnitude smaller than of typical solids, so can more easily be seen [2].

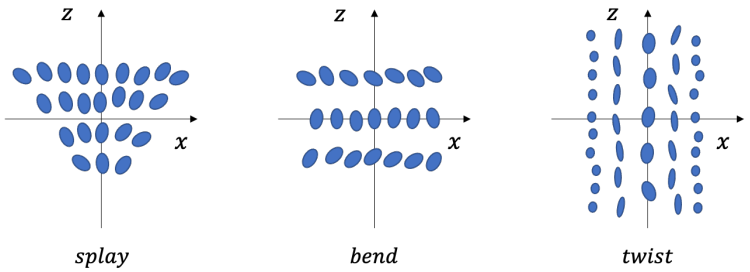


FIG. 2. Disruption of order within liquid crystals can proceed via different distortions of the molecules, shown in blue.

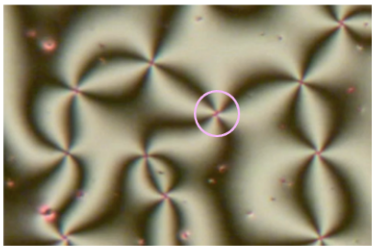


FIG. 3. With an example highlighted in a pink circle, this image shows a liquid crystal solution containing defects which have four brushes, and therefore strength of ± 1.

Topological defects are thus named as the topology of defects is preserved universally across different systems [7]. Topological arguments can be used to classify different types of defect. By specifying the director orientation of molecules at all points throughout the liquid crystal, the 'texture' of a liquid crystal can be defined [17]. This is a sort of mapping of points onto a set of allowed values of the director, which can be evaluated to classify defects [3].

Defects can be classified by positive or negative strength s [2], as the director undergoes a rotation of s multiples of 2π upon moving through a closed path within the plane [3]. The value of s is determined by the number of 'brushes' emergent from a defect, divided by four [2]; for example, in Figure 3, four brushes can be seen about a given defect, so s is equal to ± 1. As the ends of molecules are indistinguishable within liquid crystals, the director $\hat{n}$ is equivalent to $-\hat{n}$, meaning that s can take on integer and half-integer values [3]. This is known as head-to-tail invariance [7]. The sign can be determined by rotating the liquid crystal through crossed polarisers [12]; based on the director and the distribution of the molecular axes, brushes on a defect will move clockwise if it is positive, and anti-clockwise if negative [2]. An illustration of positive and negative defects is shown in Figure 4.

Umbilic defects, which have four brushes and therefore s of ± 1, behave as point defects within a 2D approximation [6]. Figure 4 shows examples of umbilic defect types which cannot be distinguishing using a polarising microscope [12]. As elastic energy density increases inversely with the square of distance towards the defect centre [3], umbilic defects have a finite isotropic core, meaning they are not true singularities [2]. However, they behave sufficiently similarly to strictly defined topological defects for study and comparison purposes [6].

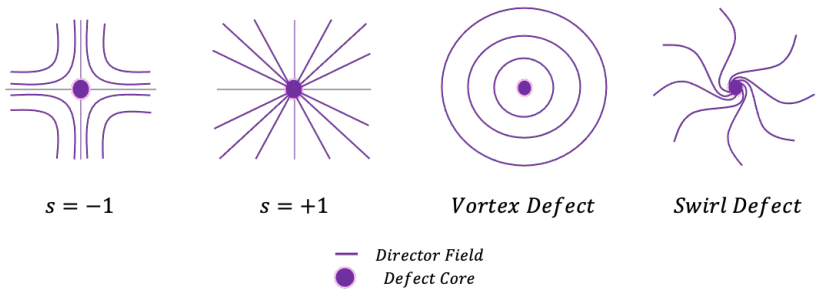


FIG. 4. Illustration of the variation of the director field about the core of a topological defect, resulting in different signs for strength classifications, as well as different types of umbilic defects known as vortex and swirl defects. Adapted from [2].

1.4 Liquid Crystals in the Presence of Electric Fields

Due to the order present within liquid crystals, certain properties can be exhibited and manipulated under the influence of an electromagnetic field [1], as the crystal undergoes a change in free energy [12]. The anisotropy of the liquid crystals described in Section 1.2 results in particular electric, magnetic, optical and elastic properties and flow behaviour in different directions, such as the dielectric constant or index of refraction [1]. For example, as the nematic phase of the liquid crystal is highly fluid and has a low viscosity, the orientation of the molecules can be changed in the presence of an electric field [2]. Upon application of the field, weak dipole moments are created as the field causes a small charge separation on the molecules. The molecules possess the same orientational order, and thus the same average dipole moment in a given direction, meaning there is a net dipole moment within the liquid crystal, known as the polarisation [1]. The polarisation changes dependent on the direction of the applied electric field relative to the director. The relative ease of polarisation switching within liquid crystal materials in the nematic phase means this is the phase most often used within technologies such as LCDs [7, 18].

As mentioned in Section 1.3, defects can be induced in the presence of an electric field. For example, using a negative dielectric material and cells with homeotropic boundary conditions (i.e. where the director is perpendicular to the cell bounding), when an electric field is applied along the director, umbilic defects are induced within a nematic liquid crystal [2]. The molecules re-orient due to the electric field when past what is known as the Fréedericksz threshold [2] [19], forming the umbilic defects.

1.5 Defect Dynamics

A continuum model assumes that defect motion is overdamped and adiabatic, with no inertia arising from motion or reorientation of the director [6]. Defects of opposite sign and equal strength within a liquid crystal tend to annihilate over time on the basis of the attractive force between them, which is inversely proportional to their separation [3]. This force results from the liquid and elastic properties of liquid crystals [2], as the director field is distorted elastically at the location of defects [20]. The long-range elastic fields of each defect in an annihilating pair cancel out [3]. In the process of annihilation, defects approach one another with increasing velocity [12], and upon contact vanish to create a uniform director field in the region [2]. This results in a decrease in the free energy of the system [20]. Not all defects within a liquid crystal undergo annihilation, as they may become pinned by impurities or irregularities present in the solution. As topological defects appear in various fields of physics, defect dynamics are widely applicable [7].

For two annihilating defects, it is observed that they have differing velocities, with the negative defect slower than the positive, and that the paths followed by the defects tend to be S-shaped [20]. The velocity arises from viscous and elastic forces, dependent respectively on director reorientation and potential gradients in the liquid crystal [6]. The dynamics

of umbilical defects induced by an electric field in two dimensions can be described by two scaling laws, one relating to the density of point defects ρ and another to the distance between annihilating defects D, both as functions of time t. The former of these relations is given as

$$\rho(t) \propto t^{-1}, \tag{3}$$

and the latter as

$$D \propto (t_0 - t)^{0.5} \tag{4}$$

where $t_0 - t$ is the time to annihilation [12]. These relationships have been verified for defects in a variety of liquid crystals [21]. These scaling laws also hold for variations in electric field frequency and amplitude, cell gap, and temperature, and are consistent with dimensional analysis of equations of motion due to the viscous forces between defects [12].

2 Machine Learning

2.1 Introduction

Machine learning is a field which has broad applicability within scientific disciplines, involving teaching computers to sort and analyse data to create outputs using algorithms produced by training on pertinent datasets [22]. Generated using statistical methods, these outputs are typically detections, predictions or classifications, dependent on the desired function of a given model [23]. The field has exploded in recent years, becoming more and more intrinsic to everyday modern technologies. Founded with more humble origins, machine learning is typically considered to have originated as a field when psychologist Frank Rosenblatt built a machine able to interpret analog signals to recognise letters of the alphabet, finding inspiration from the human nervous system [24]. Initial development of machine learning algorithms emerged as an attempt to mimic biological neural networks, thus inspiring the computing systems known as artificial neural networks, which essentially work to recognise patterns in order to intelligently analyse data for applications such as object detection. In the modern day, machine learning has been revolutionary for the processing of scientific 'big data', from analysis of biological proteins to data from the Large Hadron Collider [25].

2.2 Computer Vision

Computer Vision is a branch of machine learning whereupon computers are trained on inputs made up of images . This can be done using bounding boxes in order to detect and classify objects within an image [26]. Firstly, the model is trained using images which contain labelled objects [27]. The model then identifies objects in further input data and sorts them into classes dependent on what it has been trained to identify [28]. It makes a prediction of where the item is located via a bounding box as a set of co-ordinates surrounding the object, as



FIG. 5. Example of a bounding box for object detection. The red box surrounds a detected object, with annotation of object number (38), class (4-fold defect), and confidence level (0.65).

shown in Figure 5, and gives a percentage measure of how confident it is that the prediction is correct [26]. This process is known as object detection.

Beyond standard object detection, computer vision models have been developed that can track objects over time. After the object detection stage, the code will apply further analyses to decide if the same object is successfully being tracked. For example, the model may detect an object within the first frame. It may then predict that the object will appear in or near to the same location in the next frame, estimating via something known as a classifier [26]. This can then be compared to the independent object detection ran over the next frame. A classifier may, for example, be set to track an object given that it appears within the same approximate location for at least 5 frames in a row, without skipping more than 3 consecutive frames of object detection. This process updates continually with new frames [29].

2.3 Evaluation Metrics

In order to evaluate computer vision models, it is important to consider what metrics may be useful. The first points of evaluation to consider concern precision P versus recall R. Precision measures how many of the predictions that the model makes are correct. Recall, however, measures how many predictions that the model makes versus how many it should have made based on ground truth training data [28]. These are both very important metrics to measure the success of the model, yet there is a payoff in that increased precision may lead to lower recall and vice versa. Therefore, making a precision-recall curve as shown in Figure 6 can help to select the point at which both metrics are optimally maximised [27]. This can also be done using what is known as an $f1$ score [30], given as

$$f1 = 2 * \frac{P * R}{P + R}. \tag{5}$$

The $f1$ score should be as high as possible.

Dependent on the desired use of the model, confidence levels can be utilised to make better predictions. For example, take a model where it is not as important to identify every object as it is to identify them correctly. In this case, it would be suitable to set a high confidence threshold, meaning the model should only make a prediction when it is very sure it will be correct [28]. This is equivalent to saying that the predictions should be more precise yet have lower recall.

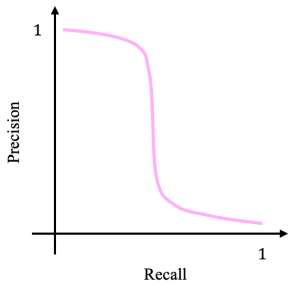


FIG. 6. As recall increases, there is a loss in precision; the model makes more predictions, and therefore it is more likely that some predictions are incorrect.

The average precision (AP) summarises the precision recall-curve of a model. Given that for a number of thresholds n

$$P(n) = 1 \tag{6}$$

and

$$R(n) = 0, \tag{7}$$

average precision can be found using the following equation [27]:

$$AP = \sum_{k=0}^{k=n-1} P(k)[R(k) - R(k+1)]. \tag{8}$$

This means it is therefore a weighted mean of precisions at a given threshold, using the increase in recall at the prior threshold as a weight [28]. To understand these thresholds, the concept of intersection over union (IoU) must be introduced. This is a measure of the success of the object detection via comparison between a predicted bounding box and a so-called 'ground truth' bounding box, i.e. one that is labelled within the training data [31]. IoU is calculated as

$$IoU = \frac{A_i}{A_u}, \tag{9}$$

where A_i is the area where the predicted and ground truth bounding boxes overlap and A_u is the total area of both bounding boxes [32]. For example, an IoU of 0.65 means a 65% overlap of the predicted and ground truth bounding boxes. Therefore, the higher the IoU, the better the prediction that the model has made. A threshold in this context is a set value of the IoU that must be met for the model to have been considered to have successfully detected an object [27].

To incorporate different thresholds into an evaluation, the metric of mean average precision (mAP) is introduced; this has become a standard metric to compare object detection softwares and models [28]. mAP is calculated by finding the AP for all classes and taking the mean,

$$mAP = \frac{1}{n} \sum_{k=1}^{k=n} AP_k \tag{10}$$

where k is a given class. The higher the mAP, the greater the accuracy of the model [27].

Loss scores are also used to evaluate the success of an object detection model [33]. For example, in YOLOv5 (see Section 2.4), box loss measures how successfully bounding boxes locate the centres and areas of objects, class, or CLS, loss measures to what degree a model misclassifies objects, and object loss gives a score of how probable it is that an object resides within a predicted bounding box [34].

2.4 YOLOv5

YOLO (You Only Look Once) is an open source computer vision architecture, thus named because it examines a full image only once using a single neural network to detect objects [26, 33]. Compared to other computer vision softwares, which instead use classifiers and analyse an image multiple times before making bounding box predictions, YOLO uses convolutional layers to analyse which predicted bounding box best detects an object [35]. This is a much faster process, making YOLO suitable for object tracking in real time. YOLO version 5 (YOLOv5) is the first implementation entirely in PyTorch rather than Darknet; this framework is more instinctive to use and runs very fast [36]. It utilises cross-section partial networks (CSP) within CSPNet in order to analyse image features, which has very efficient processing times [34]. There are different model sizes of YOLOv5, such as YOLOv5s (small), YOLOv5m (medium), and YOLOv5l (large) [36]. These differ by number of model layers and number of parameters, but ultimately function in the same way. The YOLOv5 model is comprised of a so-called backbone, neck and head. The backbone uses the CSP to identify features of the inputted image, the neck allows the model to make scaling comparisons of features of different sizes across images, and the head creates the output bounding boxes with associated metrics [34].

3 Method

3.1 Initial Plan

The initial plan for this project was to create cells containing liquid crystal samples and induce defects within them before tracking the defect paths using machine learning in order to analyse their dynamics, and compare findings to computer simulations. However, due to time constraints, it was decided that we would utilise the videos of cells created previously by Juhan Raidal, a Student Experience Intern at the University of Manchester. As well as this, greater focus was placed on analysis of the impact of nanoparticle doping within liquid crystals on the universal scaling law given by Equation (4).

3.2 Production of Data Sources

3.2.1 Making Cells

To study topological defects requires experimental sources: for our project, these were videos of cells showing defect motion towards annihilation over time, created by Juhan Raidal. Using a nematic liquid crystal within a cell forms a quasi-2D system, which allows for simpler analysis of the defects [6]. The liquid crystal used was N-(4-Methoxybenzylidene)-4-butylaniline (MBBA), with a molar mass of 267gmol^{-1}. As discussed in Section 1.4, an electric field is required to re-orient the liquid crystal and create topological defects, meaning that the inner cell walls require a conducting layer. In this case, indium tin oxide (ITO) was used over glass, which has the benefit of being transparent so the crystal can easily be seen and recorded over time. As well as this, caution was taken to ensure that the cell gap is uniform and consistent across all cells, and of the order of microns. Otherwise it may affect the results, as larger cell gaps result in fewer topological defects being formed.

3.2.2 Liquid Crystal Solutions

The parameters varied within this experiment are nanoparticle size and concentration within the liquid crystal. Therefore, three sets of 0.5% isopropanol solutions containing barium titinate nanoparticles of density $6\,02 \text{gcm}^{-3}$ were made by Juhan Raidal to combine with the MBBA: varying particle size, varying concentration of nanoparticles of a given size, and varying particle size and concentration for a constant mass concentration. The particles are approximated as spheres with constant density and diameter equal to particle size. A summary of particle sizes and concentrations for the three sets of solution is given in Table 1. Five cells were used for each set. To avoid aggregation of nanoparticles in solution, which might overly affect defect formation, only small amounts of nanoparticles were added. An additional cell with no nanoparticles was also created and recorded. It should be noted that the nanoparticles are 3 orders of magnitude smaller than the scale at which they are observed.

3.2.3 Inducing Defects in Liquid Crystal Solutions

The nematic liquid crystal materials in the cells had a negative dielectric anisotropy. Therefore, the orientation of the molecules changed in the presence of an electric field,, as discussed in Sections 1.3 and 1.4. This produced umbilic defects. In order to study the defect dynamics, the cells needed to be recorded over time. The application of the electric field and the recording of subsequent motion was done in the laboratory by Juhan Raidal using a microscope with a camera attachment and a polariser. To allow for calibration between pixels and microns, the scale of the cells and at which the defects were visible, images of 2048 by 1088 pixels were taken of a micron ruler at the same magnification as the cells.

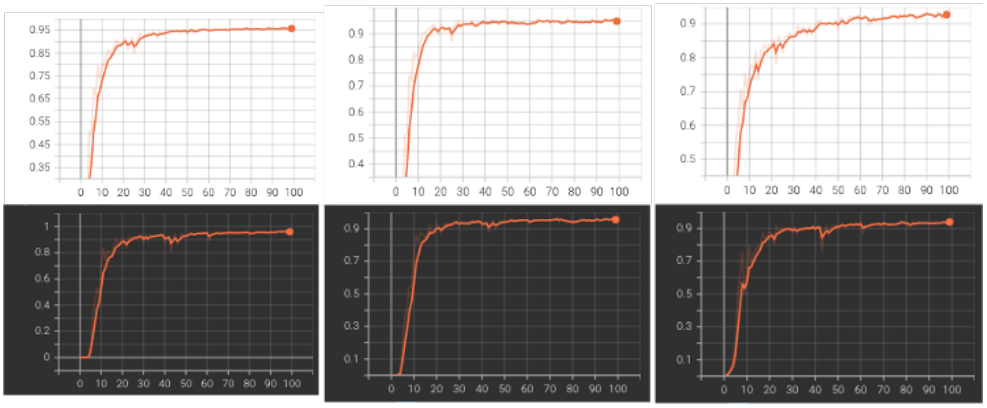
	Particle Size/nm	Mass Conc./%	Particle Conc./%
Vary Particle Conc.	280	0.01	0.00006
	280	0.05	0.0003
	280	0.1	0.0006
	280	0.2	0.001
	280	0.4	0.003
Vary Particle Size	45	0.00015	0.0002
	90	0.0012	0.0002
	280	0.035	0.0002
	370	0.081	0.0002
	500	0.2	0.0002
Vary Particle Conc. and Size	45	0.1	0.15
	90	0.1	0.019
	280	0.1	0.0006
	370	0.1	0.0003
	500	0.1	0.0001

TABLE 1. Summary of particle sizes, mass concentrations and particle concentrations (conc.) used and investigated in liquid crystal solutions. Particles have been approximated as spheres of constant density with diameter equal to their size.

3.3 Computer Vision and Model Training

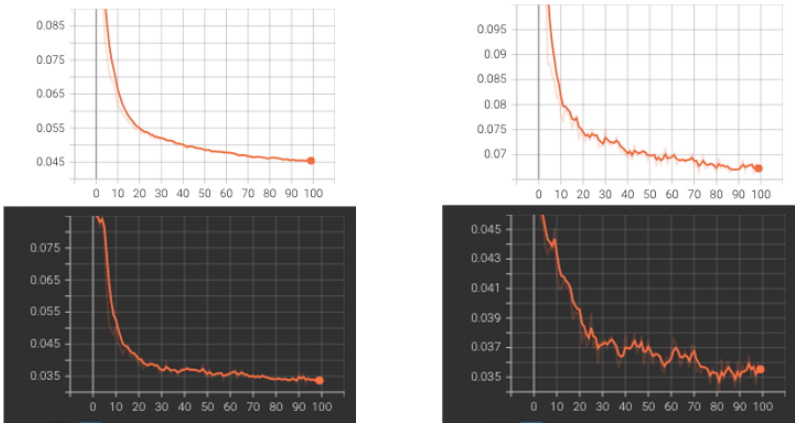
3.3.1 Image Extraction and Training Data

A training dataset of images is required to train an object detection model. Images of umbilic defects were therefore collated by Juhan Raidal from online sources and previous research by Yuah Chen and Jason Domingues, summing to 380 images. The images were then uploaded onto online platform Roboflow, in order to use the Roboflow Annotate tool; this allowed each defect to be classified manually labelled with a bounding box. Within Roboflow, the training images were then augmented and preprocessed to provide the model with more data; a random selection of images were rotated, flipped, or split into four.



(a) mAP (b) Precision (c) Recall

FIG. 7. Top and bottom row graphs respectively demonstrate models trained on initial and re-labelled data, with y-axes denoting a given metric and x-axes number of epochs. Precision, and mAP at an IoU threshold of 0.5, are better for the bottom row model, while recall is better for the top row model.



(a) Box loss (b) Object loss

FIG. 8. Comparing models for object detection of topological in liquid crystals, where x-axes denote number of epochs. Bottom row graphs depict a model trained on re-annotated data with tighter, more accurate bounding boxes than for the model in the top row. There is greater box loss and object loss, respectively shown in (a) and (b), for the model depicted in the graphs in the top row. Note that both models only detect one class of object, meaning class loss is zero.

While Juhan Raidal had previously annotated the training dataset, the defects were relabelled with smaller bounding boxes that were better centred upon the defects. This was to create more precise tracking of defects and obtain more accurate values for defect co-ordinates. Testing the models produced by YOLOv5 on different versions of the labelled dataset over 100 epochs (number of cycles through the training dataset undertaken by the model) demonstrated that the tighter bounding boxes were more suitable for purpose within this project. Analysis metrics, as discussed in Section 2.3, demonstrated that while fewer predictions were made by the model, they were more likely to correspond accurately and precisely to the ground truth bounding boxes. Metric graphs for both models related to mAP, precision and recall are provided in Figure 7, while loss metrics are shown in Figure 8.

3.3.2 Inference and Object Tracking

The model weights were therefore produced using the relabelled training dataset with YOLOv5s (chosen for fast inference times) in Roboflow. The so-called 'best' weights were extracted which optimised precision vs. recall as in Equation 3, and were then used to analyse videos of defects produced in cells detailed within Section 3.2.1. These videos were sped up by three times to reduce runtime, as there were a very large number of data points (i.e. frames, given that the videos were originally 30 frames per second and around 3 minutes long). They were also cropped, to remove sections prior to defect formation.

DeepSORT (Simple Online and Realtime Tracking with a Deep association metric) is an object tracking software which functions using methods as described in Section 2.2 [37]. A notebook in Google Colab was used with DeepSORT in order to track the defects in videos of the cells, analysing on a per-frame basis and producing further videos showing bounding boxes on the defects with confidence levels. An image from such a video is shown in Figure 9.

Defect co-ordinate data with time ordering from a given video was compiled into a text file as a result of the detection and tracking made using the YOLOv5 and DeepSORT algorithms. Output text files contained the leftmost x and uppermost y co-ordinates of the bounding boxes in a given frame, as well as width and height of the bounding box, measured in pixels. A code was written to read the text files and convert these values into the central x and y co-ordinates of each bounding box, corresponding to the location of defects. By observing the video with tracked defects, appropriate defects within an annihilating pair could be selected; after input of their defect labels, shown in the video, the code calculated the distance between input defects d at each frame, using the equation

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \tag{11}$$

where (x_1, y_1) and (x_2, y_2) are the central co-ordinates of the two annihilating defects. As there were 10 frames per second, the number of frames was divided by ten to convert into seconds. By examining the text file and using the micron ruler, it was also calculated that a conversion factor of 0.755 was necessary to convert pixels into microns as desired.

The defect separation array was also reversed, meaning that output graphs demonstrate time to annihilation as desired within the scaling law being investigated, Equation (4). Therefore, the code gave output arrays of defect separation in microns and time to annihilation in seconds. Secondary code was also written which took inputs from the text file to produce preliminary graphs showing the paths of annihilating defects by plotting the arrays of the x and y central co-ordinates of the bounding boxes (see Section 4.2.5).

3.3.3 Video Selection and Exponent Extraction

Taking logarithms allows for relations between distance between annihilating defects and time to annihilation to be expressed in the form of a linear graph. On the basis of Equation (4), by plotting double-logarithm graphs of defect separation over time to annihilation, the linear gradient of the graph is equal to the exponent of the scaling law. More code was written which took linear fits over the range of 3.8 to 4.5 logarithmic units, as this seemed to consistently provide good linear fits across different defect pairs and solutions. This will be further discussed in Section 4.1. Exponents were only deemed suitable when a good linear

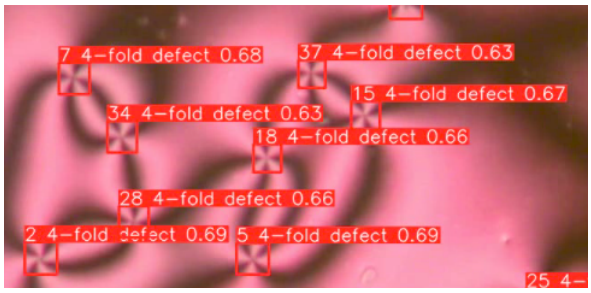


FIG. 9. A snapshot at a given frame of object tracking of defects within a liquid crystal solution, showing bounding boxes, defect numbers, class label and confidence levels.

relationship was found. Video data of cells was also discarded due to effects such as dust within cells and limited number of defect pairs; notably, sped-up videos that fell beneath a length of one minute tended to provide poor results, failing to produce sufficient data for exponent extraction. For example, videos taken at 0.05% mass concentration for a constant particle size of 280nm were discarded due to the visible dust within the cells. The dust had a notable effect on the behaviour of defect pairs, as the linear relationships for annihilating defects within these videos were poor.

4 Results

4.1 Errors

The first source of uncertainty presented itself via consideration of the bounding boxes. By watching videos with object tracking on stationary or slow-moving defects, it became visually apparent that there were small fluctuations in the position of bounding boxes over time. Therefore, a video that showed semi-stationary defects in a solution without nanoparticle doping was selected. Considering one of these defects when stationary over a time interval of 3 seconds (i.e. 1 second of the sped-up video), the central co-ordinates of the bounding boxes produced at each frame were then extracted. Taking the difference of maximum and minimum values of x or y values and dividing by two, and averaging across different stationary defects, gave an initial absolute error estimation in x and y. This was then propagated as before to give an error in defect distance for each data point. Plotting this on a graph demonstrated that the error was approximately equivalent to statistical jitter, which seems as would be expected, as the jitter largely arises due to uncertainty from the machine learning process.

Taking a linear fit was done by doing a least-squares regression of the data points using SciPy, a Python library. This produced a covariant matrix whereupon the square root of the diagonals gave an error in the gradient and intercept of the fit. However, while finding errors in x and y co-ordinates as previously discussed was useful for calculating the linear fit, the

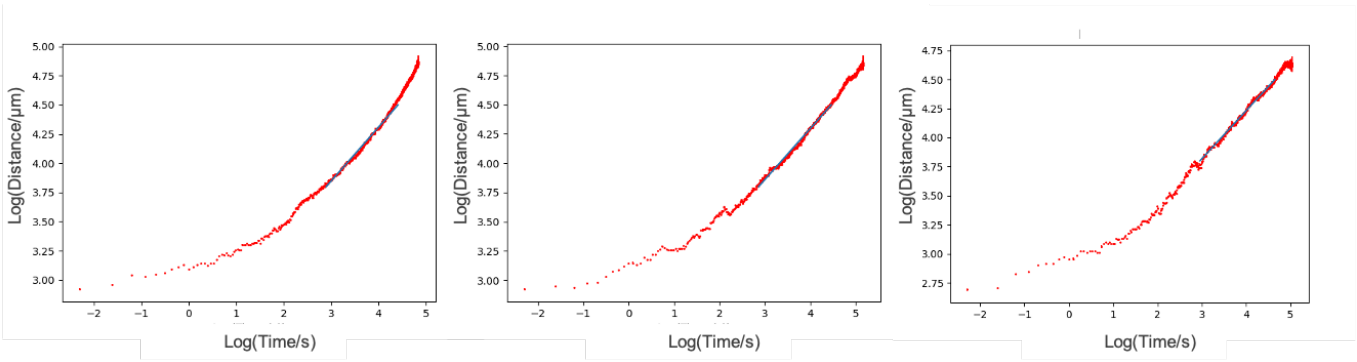
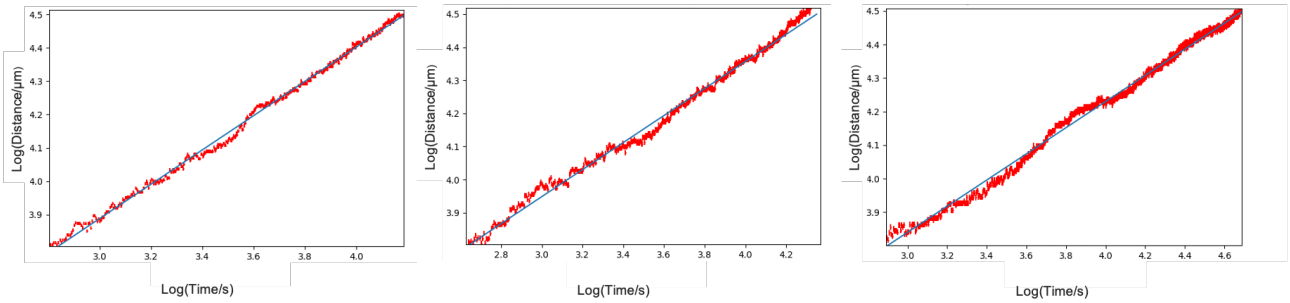


FIG. 10. The graphs depict linear relationships between logarithms of defect distance and time to annihilation. Linear fits in blue have been made in the comparable range of 3.8-4.5 microns where the red datapoints are well described by a linear relationship.



(a) 280nm, 0.01% (b) 90nm, 0.0012% (c) 500nm, 0.1%

FIG. 11. Examples of distance-time data for annihilating defects within a logarithmic distance range of 3.8-4.5. Graphs are labelled with nanoparticle size and mass concentration. Error in distance is multiplied by two for visibility. Scaling exponent from a linear fit of graph (a) has value $\alpha = 0.514 \pm 0.025$. Graph (b) produces $\alpha = 0.427 \pm 0.025$. Graph (c) gives $\alpha = 0.390 \pm 0.025$.

errors provided by SciPy for the gradient were too small to attribute to our exponent values. It became apparent that the variation across exponents for defect pairs at a given particle size and concentration was not covered by this error.

To account for this, a middling nanoparticle concentration of 0.10% and nanoparticle size of 280nm was chosen to analyse the variation in exponents and linear fits. The distance-time graphs for three defect pairs, from three different videos, were compared and it was decided that they all were well described with a linear fit across a range of 3.8-4.5 logarithmic units, allowing consistent comparison, as shown in Figure 10. Therefore, all fits were taken across this range for defect pairs. The difference in the exponents produced for the pertinent three defect pairs was divided by two to give an absolute error for each extracted exponent value. This error came out as 0.025 units. When multiple suitable exponents were found for a given concentration and particle size, a weighted average was taken to provide an exponent value and associated error. As each individual measurement had the same error , the typical standard error σ of

$$\sigma = \frac{0.025}{\sqrt{N}} \tag{12}$$

was used, where N is equal to the sample size (i.e. the number of exponents at a given size or concentration). N varied as different videos yielded different numbers of defect pairs that were suitable for analysis, due to various factors such as cell gap variability or dust within the cells discussed in Section 3.3.3.

4.2 Graphs and Data Fits

4.2.1 Scaling Exponents

The method described in Section 3.3.3 to extract scaling exponents at various particle sizes, mass concentrations and particle concentrations produced scaling exponent values on the basis of graphical linear fits in a region of 3.8-4.5. A sample of resultant graphs, and their extracted exponent values, can be found in Figure 11.

In order to test our methodology for finding the scaling law exponent, it was performed on a cell which contained no nanoparticles, resulting in an exponent α of $\alpha = 0.517 \pm 0.018$. This value is within one standard deviation of the value given in Equation (4), $\alpha = 0.5$, and thus was taken as a confirmation of the validity of our experimental method.

4.2.2 Varying Nanoparticle Concentration

Figure 12 shows the output graph of scaling exponent against nanoparticle mass concentration. In this case, as particle size is constant, as the mass concentration increases the particle concentration also increases. The trend seems to clearly show that an exponential decay fit is suited to the data. Therefore, for an exponential fit

$$y = a(1-b)^x + c, \tag{13}$$

values of a,b and c were found as follows: $a = 0.121 \pm 0.004$, $b = 0.99999996 \pm 0.00000008$ and $c = 0.400 \pm 0.003$. In this case, y is equal to the scaling exponent α for constant particle size and x is equal to mass concentration. These errors were calculating using the square root of diagonals of the covariant matrix produced by the *curve_fit* function in Scipy.

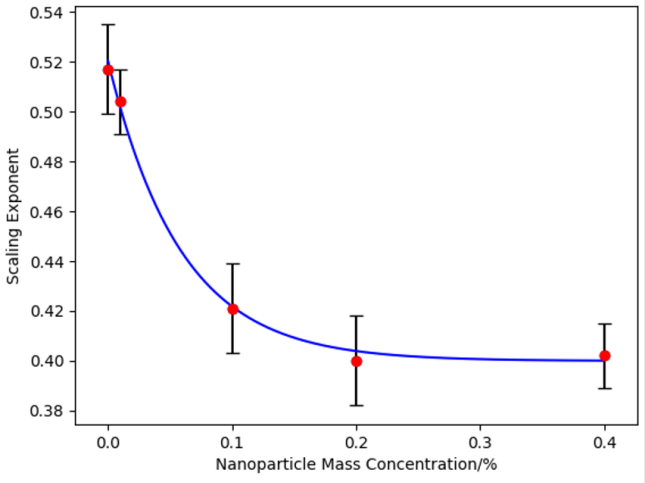


FIG. 12. Graph showing dependency of scaling exponent on particle mass concentration (and thus on particle number) for constant particle size. Data points shown in red are well described by the blue line, an exponential fit.

4.2.3 Varying Nanoparticle Size

Similarly as in Section 4.2.2, Figure 13 shows the found results of scaling exponents at different nanoparticle sizes with constant particle concentration. This again seems to be well described by exponential decay: for an exponential fit as in Equation (13), a = 0.121 ± 0.009, b = 0.024 ± 0.003 and c = 0.398 ± 0.003. However, in this case, y is the scaling exponent α at constant particle concentration, while x is equal to the particle size.

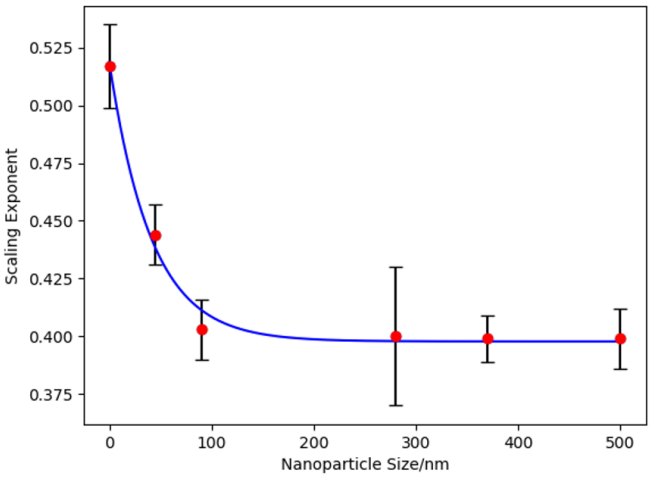


FIG. 13. Graph showing dependency of scaling exponent on particle size, for constant particle concentration. Data points shown in red are well described by the blue line, an exponential fit.

4.2.4 Varying Nanoparticle Size and Concentration

Figure 14 shows data that results from varying both the particle concentration and the particle size at a constant mass concentration. Alongside this, two data point predictions made by the previous scaling laws are included. The errors on these datapoints were propagated from the exponential fits using partial derivatives. The data is well described by an exponential decay, and the data from all three datasets appears consistent. Using the exponental fit described in Equation (13), where y is scaling exponent α at constant mass concentration and x is particle size, constants are found as a $= 0.124 \pm 0.013$, b $= 0.033 \pm 0.014$, and c $= 0.39 \pm 0.05$.

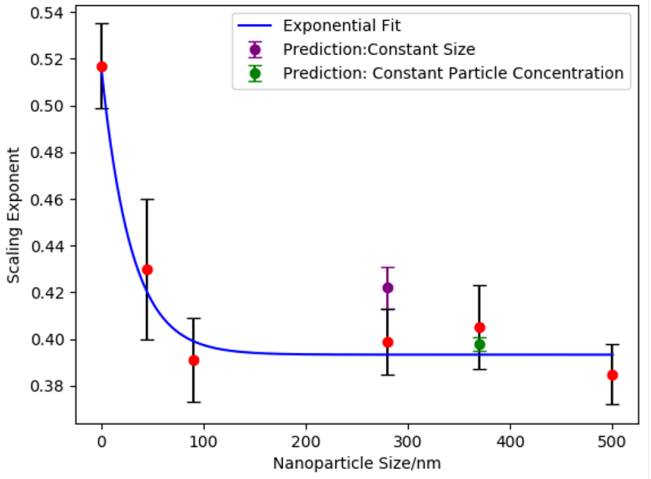


FIG. 14. Graph showing dependency of scaling exponent on particle size when particle size and particle concentration are varied, yet mass concentration is constant. Data points shown in red are well described by the blue line, an exponential fit. Purple and green data points are predictions from previous datasets.

4.2.5 Defect Paths

Within this project, we were unable to numerically analyse the defect paths due to time constraints, but can draw qualitative conclusions from graphical results. The defect path graphs produced, such as in Figure 15, consistently demonstrated S-shaped paths. They also tended to show that one defect within an annihilating pair moved more quickly than its partner, shown by the greater distance travelled within the same time period. The errors in x and y co-ordinates are those derived from stationary defects described in Section 4.1.

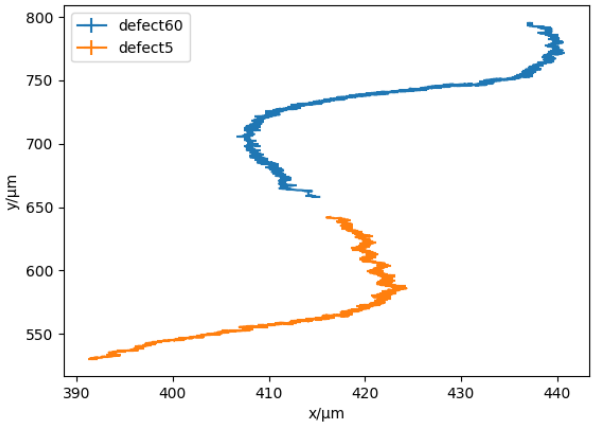


FIG. 15. An instance of a graph depicting the x and y co-ordinates of an annihilating defect pair over time, clearly exhibiting an S-shaped curve. Legend labels 'defect60' and 'defect5' denote the defect numbering provided by the object detection model. Errors are shown for every tenth data point.

5 Discussion

5.1 Interpretation of Results

The relationships found for scaling law exponent given barium titanate nanoparticle doping suggest that as the amount of nanoparticles increases, the scaling exponent decreases exponentially. Therefore, this indicates a breakdown of the scaling law in Equation (4) as nanoparticles are introduced into solution. It could also be an indication that the quasi-2D approximation for nematic liquid crystals and approximation of umbilic defects as point defects change in the presence of nanoparticles. The annihilating defects that were tracked seemed to rudimentarily demonstrate S-shaped paths and differing velocities within pairs.

5.2 Improvements and Future Research

One of the biggest constraints of this project was a lack of time. Machine learning is very useful to analyse defects in liquid crystals as it can provide a lot of co-ordinate information in order to consistently track defects across many different videos. However, the large volumes of data required to successfully train a model as well as to produce results meant that a lot

of time was spent organising and running files. Therefore, there were limits on the number of videos that could be analysed, and thus on the number of scaling exponents for given nanoparticle parameters. Extracting more exponents would help to improve final datasets and reduce the large errors as shown in Figures 12-14.

The process of tracking defects could be further automated by using machine learning to identify annihilating defect pairs, perhaps using nearest-neighbour algorithms or computer vision. Although introducing more error into the process, this could be mitigated if more automation would allow much more data to be analysed. This would also require the model to be improved. As annihilating defect pairs were identified by eye, human evaluation was used to see if the model had correctly identified defects. However, if this process was automated, a confidence level threshold would have to be introduced. Therefore, the confidence levels attributed to the bounding boxes should be increased by improving the model.

The model could be improved by drawing even tighter bounding boxes and using more training data. One concern within the project was that the model often failed to detect defects when they were close to annihilating, so more images of defects close to annihilation within the training dataset may help to mitigate this problem. The confidence levels of the model were infrequently at or over a value of 0.7, which suggests that more training data would be very useful to improve the results. This is especially noteworthy given that precision is prioritised over recall for the aims of this project, as it is more important that the defects are tracked precisely than that all of the defects are necessarily detected.

While a methodology to track defect paths was created, the paths were not systematically analysed. Therefore, it would be a suitable next step to systematically compare the shape of paths produced by defects both with and without nanoparticles within liquid crystal solutions. This experimental data could be compared with computer simulations for defect paths. A method could then be developed to calculate the speed of defects between consecutive frames in order to numerically compare the speed of different defects in an annihilating pair.

6 Conclusion

Within this project, computational work was done to track topological defect annihilation within liquid crystal solutions using machine learning algorithms. These solutions were doped with barium titanate nanoparticles of various sizes, with different mass concentrations and particle concentrations in solution. The resulting data demonstrated that the scaling exponent relating distance between annihilating defects and time to annihilation decreases exponentially as nanoparticle size and nanoparticle concentration increase. This suggests breakdown of the scaling laws in the liquid crystal due to the presence of nanoparticles, which could be extrapolated to other systems with topological defects. Confirmation was also found of S-shaped defect paths by tracking annihilating defect co-ordinates and producing graphical representations, producing a methodology for further work using machine learning to extract information on defect dynamics in liquid crystals.

References

[1] P. J. Collings and M. Herd, *Introduction to Liquid Crystals, Chemistry and Physics*. London: Taylor Francis Ltd., 1997.

[2] I. Dierking and P. Archer, "Imaging liquid crystal defects," *RSC Advances*, vol. 3, no. 26433, 2013.

[3] W. F. Brinkman and P. E. Cladis, "Defects in liquid crystals," *Physics Today*, vol. 35, no. 5, 1982.

[4] M. Mitov, "Liquid-crystal science from 1888 to 1922: Building a revolution," *ChemPhysChem*, vol. 15, no. 7, pp. 1245–1250, 2014.

[5] H. W. Chen, J. H. Lee, B. Y. Lin, *et al.*, "Liquid crystal display and organic light-emitting diode display: present status and future perspectives," *Light: Science Applications*, vol. 7, 2018.

[6] H. Kirsten and S. Ralf, "Topological point defects of liquid crystals in quasi-two-dimensional geometries," *Frontiers in Physics*, vol. 8, p. 112, 2020.

[7] M. Kralj, M. Kralj, and S. Kralj, "Topological defects in nematic liquid crystals: Laboratory of fundamental physics," *Physica Status Solidi A: Applications and Materials Science*, vol. 218, no. 17, 2021.

[8] T. W. B. Kibble, "Topology of cosmic domains and strings," *Journal of Physics A: Mathematics and General*, vol. 9, no. 8, 1976.

[9] H. Kelker, "History of liquid crystals," *Molecular Crystals and Liquid Crystals*, vol. 21, pp. 1–48, 1973.

[10] G. A. DiLisi, "History," in *An Introduction to Liquid Crystals*, 2053-2571, Morgan Claypool Publishers, 2019.

[11] D. Coates, *Electronic Materials*, ch. The Chemistry of Liquid Crystals. Boston, Massachussets: Springer, 1991.

[12] I. Dierking, O. Marshall, J. Wright, and N. Bulleid, "Annihilation dynamics of umbilical defects in nematic liquid crystals under applied electric fields," *Physical Review E*, vol. 71, June 2005.

[13] D. Bruce, R. Deschenaux, B. Donnio, and D. Guillon, "Metallomesogens," in *Comprehensive Organometallic Chemistry III* (D. Michael, P. Mingos, and R. H. Crabtree, eds.), pp. 195–293, Oxford: Elsevier, 2007.

[14] I. Carlescu, "Introductory chapter: Nematic liquid crystals," in *Liquid Crystals and Display Technology* (M. S. Ghamsari and I. Carlescu, eds.), IntechOpen, 2020.

[15] I. Dierking, *Textures of Liquid Crystals*, ch. 1: Introduction, pp. 1–20. John Wiley and Sons, Ltd, 2003.

[16] A. B.-R. 1 and H. T. Diep, "Nematic and smectic phases: Dynamics and phase transition," *Symmetry*, vol. 12, September 2020.

[17] H. R. Wenk and P. V. Houtte, "Texture and anisotropy," *Reports on Progress in Physics*, vol. 67, pp. 1367–1428, July 2004.

[18] J. C. Jones in *Handbook of Optoelectronics: Enabling Technologies* (J. Dakin and R. Brown, eds.), ch. Liquid Crystal Displays, pp. 137–224, Boca Raton: Taylor Francis Group, 2018.

[19] C. Cirtoaje, "Ferroelectric particles in nematic liquid crystals with soft anchoring," *Molecules*, vol. 26, 2021.

[20] Y. Shen and I. Dierking, "Annihilation dynamics of topological defects induced by microparticles in nematic liquid crystals," *Soft Matter*, vol. 43, 2019.

[21] R. R. Guimarães, R. S. Mendes, P. R. G. Fernandes, and H. Mukai, "Annihilation dynamics of stringlike topological defects in a nematic lyotropic liquid crystal," *Journal of Physics: Condensed Matter*, vol. 25, April 2013.

[22] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *SN Computer Science*, vol. 2, no. 160, 2021.

[23] S. Shalev-Shwartz and S. Ben-David, *Understanding Machine Learning: From Theory to Algorithms*. Cambridge: Cambridge University Press, 2014.

[24] A. L. Fradkov, "Early history of machine learning," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 1385–1390, 2020.

[25] T. Hey, K. Butler, S. Jackson, and J. Thiyagalingam, "Machine learning and big scientific data," *Philosophical Transactions A*, vol. 378, 2020.

[26] Y. Marushko, "Computer vision: Tracking and detecting moving objects." Website: https://dzone.com/articles/computer-vision-tracking-and-detecting-of-moving-o, October 2019. Accessed: 18-12-2021.

[27] A. F. Gad, "Evaluating object detection models using mean average precision (mAP)." Paperspace Blog. Website: https://blog.paperspace.com/mean-average-precision/, 2020. Accessed: 29-12-2021.

[28] J. Solawetz, "What is mean average precision (mAP) in object detection?." Roboflow. Website: https://blog.roboflow.com/mean-average-precision/, May 2020. Accessed: 29-12-2021.

[29] M. Broström, "Real-time multi-object tracker using yolov5 and deep sort." Github: https://github.com/mikel-brostrom/Yolov5_DeepSort_Pytorch, 2020.

[30] D. J. Hand and P. C. . N. Kirielle, "F*: an interpretable transformation of the f-measure," *Machine Learning*, vol. 110, no. 2, pp. 451–456.

[31] H. Rezatofighi, N. T. J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 658–666, 2019.

[32] M. A. Rahman and Y. Wang, "Optimizing intersection-over-union in deep neural networks for image segmentation," in *Advances in Visual Computing* (G. Bebis, R. Boyle, B. Parvin, D. Koracin, F. Porikli, S. Skaff, A. Entezari, J. Min, D. Iwai, A. Sadagic, C. Scheidegger, and T. Isenberg, eds.), (Cham), pp. 234–244, Springer International Publishing, 2016.

[33] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection." Allen Institute for AI, Facebook AI. University of Washington. arXiv:1506.02640, 2015.

[34] M. Patel, "YOLOv5 — explained and demystified," *Towards AI – Multidisciplinary Science Journal*, July 2020. https://towardsai.net/p/computer-vision/yolo-v5%E2%80%8A-%E2%80%8Aexplained-and-demystified. Accessed: 28-12-2021.

[35] U. Michelucci, *Advanced Applied Deep Learning: Convolutional Neural Networks and Object Detection*. New York: Apress, 1 ed., 2019.

[36] J. Solawetz and J. Nelson, "How to train YOLOv5 on a custom dataset." Roboflow. Website: https://blog.roboflow.com/how-to-train-yolov5-on-a-custom-dataset/, June 2020. Accessed: 29-12-2021.

[37] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric." University of Koblenz-Landau, Queensland University of Technology. 10.1109/ICIP.2017.8296962., 2017.

A Code: distance and co-ordinate arrays

```
import numpy as np

# Arrays
row_separated = []
frame_count = []
defect_label = []
elements = []
x1_coords = []
x2_coords = []
y1_coords = []
y2_coords = []
error_x1 = []
error_x2 = []
error_y1 = []
error_y2 = []

# Open text file and store an array of each of the elements of a row
with open('Clean_cell_2_5V_1.txt', 'r') as f:
    lines = f.readlines()
    the_xs = []
    the_ys = []
    for line in lines:
        row_separated.append(line.split(' '))

# Compute the central position of bbox of each defect in a pair
def defect(pair):
    x = []
    y = []
    w = []
    h = []
    all_x = []
    all_y = []
    all_w = []
    all_h = []
    all_bbox_c = []
    for row in row_separated:
        #Compute x and y of certain pair
        if row[1] == pair:
            all_x.append(row[2])
            all_y.append(row[3])
            all_w.append(row[4])
            all_h.append(row[5])
        xywh = list(zip(all_x,all_y,all_w,all_h))
    for i in xywh:
        x = float(i[0])
        y = float(i[1])
        w = float(i[2])
        h = float(i[3])
        xc = float(x + (w / 2))
        yc = float(y - (h / 2))
        bbox_c = xc, yc
        all_bbox_c.append(bbox_c)
    return all_bbox_c

# Ask for the label of each defect in a pair
```

FIG. 16. Section I of code designed to analyse text files produced by DeepSORT and YOLOv5 algorithms on the model trained to detect and track defects in liquid crystal cells. The code converts values in the text file into central co-ordinates of the bounding boxes, and thus of the defects, and compounds values into arrays.

```
# Ask for the label of each defect in a pair
first = input("Enter the label of the first defect \n")
bbox_first = defect(first)
second = input("Enter the label of the second defect \n")
bbox_second = defect(second)

# Function that converts into correct units
def correct_units(number_of_pixels):
    microns = number_of_pixels*0.755
    return microns

# Compute centre of each defect and the distances betwen them
xy1xy2 = list(zip(bbox_first,bbox_second))
distances = []
error_distances = []
for coord in xy1xy2:
    # x and y coordinates of defect 1 and 2
    x1 = correct_units(coord[0][0])
    x1_coords.append(x1)
    x2 = correct_units(coord[1][0])
    x2_coords.append(x2)
    y1 = correct_units(coord[0][1])
    y1_coords.append(y1)
    y2 = correct_units(coord[1][1])
    y2_coords.append(y2)
    # Computing the distance between defect 1 and 2
    xdif = x1 - x2
    ydif = y1 - y2
    xsquare = (xdif)**2
    ysquare = (ydif)**2
    distance = np.sqrt(xsquare + ysquare)
    distances.append(distance)
    error_distance = np.sqrt(0.205767*xsquare+0.3766464*ysquare)
    error_distance = error_distance/((distance**2))
    error_distances.append(error_distance)

# Create distances file
distances = ' '.join(str(e) for e in distances)
error_distances = ' '.join(str(e) for e in error_distances)
file_name = "distances_" + first + "_and_" + second + "_pair"
file = open(file_name, "x")
file.write(distances + "S" + error_distances)
file.close()

# Create x1, x2, y1, y2 coordinates file
x1_coords = ' '.join(str(e) for e in x1_coords)
x2_coords = ' '.join(str(e) for e in x2_coords)
y1_coords = ' '.join(str(e) for e in y1_coords)
y2_coords = ' '.join(str(e) for e in y2_coords)
file2_name = "coordinates_" + first + "_and_" + second + "_pair"
file2 = open(file2_name, "x")
file2.write(x1_coords + "S" + x2_coords + "S" + y1_coords + "S" + y2_coords)
file2.close()
print("finished!")
```

FIG. 17. Section II of code designed to analyse text files produced by DeepSORT and YOLOv5 algorithms on the model trained to detect and track defects in liquid crystal cells. The code asks for defect numbers as input, which can be found by visual analysis of videos with defect tracking. The distance between these defects is then calculated in microns and compiled within an array. Errors associated with bounding box location are similarly compiled. Ultimately, the code produces an output of text files, one with the distance arrays and another with co-ordinate arrays.

B Code: Defect Separation versus Time to Annihilation logarithmic graphs

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Ask for the label of each defect in a pair
first = input("Enter the label of the first defect \n")
second = input("Enter the label of the second defect \n")

# Open text file with the distances between the defects in a pair stored
file_name = "distances_" + first + "_and_" + second + "_pair"
with open(file_name, 'r') as f:
    distances = f.read()

# Store the distances and the error in the distances from text file in arrays
distances = distances.split("S")
error_distances = np.array(distances[1].split()).astype(float)
distances = np.array(distances[0].split()).astype(float)


# Linear fit equation
def linear(x,m,c):
    y = m*x+c
    return y

#---Graph of distance between defects in a pair vs time to annihilation---
# X (time)
frames = list(range(len(distances)))
fps=10
x = frames[1:]
for i in range(len(x)):
    x[i] = x[i]/fps
x_log = np.log(x)
# Y (distance)
y = list(reversed(distances))[1:]
y_log = np.log(y)

#----Normal plot----
'''
plt.plot(x, y)
#plt.errorbar(x, y, yerr=ey)
plt.xlabel('time (s/10)')
plt.ylabel('defect distance (microns)')
plt.title('Distance vs time')
plt.show()
'''

#----Linear fit plot----
ploty=[]
plotx=[]
ploterr = []
miny = 3.8
maxy = 4.5

yerr = error_distances[1:]
```

FIG. 18. Section I of code written to produce logarithmic graphs of defect separation against time to annihilation. Inputting defect numbers consistent with those used to create distance arrays (see Appendix A) results in re-ordering of array values into the desired format and units. A linear fit function is defined here.

```
x = frames[1:]
for i in range(len(x)):
    x[i] = x[i]/fps
x_log = np.log(x)
# Y (distance)
y = list(reversed(distances))[1:]
y_log = np.log(y)

#----Normal plot----
'''
plt.plot(x, y)
#plt.errorbar(x, y, yerr=ey)
plt.xlabel('time (s/10)')
plt.ylabel('defect distance (microns)')
plt.title('Distance vs time')
plt.show()
'''

#----Linear fit plot----
ploty=[]
plotx=[]
ploterr = []
miny = 3.8
maxy = 4.5

yerr = error_distances[1:]

for i in range(len(y_log)):
    if (y_log[i] > miny) & (y_log[i] < maxy) :
        ploty = np.append(ploty, y_log[i])
        plotx = np.append(plotx, x_log[i])
        ploterr = np.append(ploterr, yerr[i])


fit = curve_fit(linear, xdata = plotx, ydata = ploty, p0 = [0.5, 3], sigma = ploterr)


print(fit)
fit_err = fit[1]
fit_error = np.sqrt(np.diag(fit_err))
print(fit_error)

xstart = (miny-fit[0][1])/(fit[0][0])
xend = (maxy-fit[0][1])/(fit[0][0])


fig1 , ax1 = plt.subplots()
ax1.errorbar(x_log,y_log,yerr = yerr*2, fmt = 'ro', errorevery = 10, ms = 1, zorder = 1)
ax1.set_xlabel('Log(Time, (s))')
ax1.set_ylabel('Log(Defect pair separation, (μm))')
#ax1.set_title('Distance vs Time')

ax1.plot([xstart, xend], [miny, maxy], zorder = 2)
plt.show()
```

FIG. 19. Section II of code written to produce logarithmic graphs of defect separation against time to annihilation. Values are plotted to produce a graph, and a linear fit of the values is taken within a suitable regime.

C Code: defect pair tracking graph

```
import numpy as np
import matplotlib.pyplot as plt


# Ask for the label of each defect in a pair
first = input("Enter the label of the first defect \n")
second = input("Enter the label of the second defect \n")

# Open text file with the distances between the defects in a pair stored
file_name = "coordinates_" + first + "_and_" + second + "_pair"

with open(file_name, 'r') as f:
    coordinates = f.read()

# Store the x and y coordinates from text file in arrays
coordinates = coordinates.split("S")
x1_coords = np.array(coordinates[0].split()).astype(float)
x2_coords = np.array(coordinates[1].split()).astype(float)
# Flipping the y coordinates to account for the fact that it is counted from the top
y1_coords = (820-(np.array(coordinates[2].split()).astype(float)))
y2_coords = (820-(np.array(coordinates[3].split()).astype(float)))

# Convert errors from pixels to micrometres
def correct_units(number_of_pixels):
    microns = number_of_pixels*0.755
    return microns

xerr = correct_units(0.425)
yerr = correct_units(0.575)


#----------Graph of paths of each defect-----------
plt.errorbar(x1_coords,y1_coords, yerr=yerr, xerr=xerr, errorevery = 10, label="defect" + first)
plt.errorbar(x2_coords, y2_coords, yerr=yerr, xerr=xerr, errorevery = 10, label="defect" + second)
plt.xlabel('x/μm')
plt.ylabel('y/μm')
#plt.title('Path of defect pair')
plt.legend()
plt.show()

```

FIG. 20. The code shown here uses co-ordinate files produced by code from Appendix A and creates a graphical plot of the x and y co-ordinates in microns, with associated error from bounding box considerations.

D Exponent values

Mass Concentration/%	0	0.01	0.1	0.2	0.4
Particle Concentration/%	0	0.00006	0.0006	0.001	0.003
Exponents	0.523	0.516	0.438	0.311	0.441
	0.511	0.514	0.405	0.481	0.388
		0.544			0.347
		0.443			0.432
Standard Error	0.018	0.013	0.018	0.018	0.013

TABLE 2. Exponent values, each from a different defect pair, for varying nanoparticle concentrations at a constant particle size of 280nm. Each individual exponent has an associated error of ± 0.025 before propagation.

Particle Size/nm	0	45	90	280	370	500
Mass Concentration/%	0	0.00015	0.0012	0.035	0.081	0.2
Exponents	0.523	0.425	0.389	0.400	0.381	0.403
	0.511	0.435	0.388		0.403	0.395
		0.465	0.408		0.353	
		0.452	0.427		0.399	
					0.472	
					0.389	
Standard Error	0.018	0.013	0.013	0.03	0.01	0.013

TABLE 3. Exponent values, each from a different defect pair, for varying nanoparticle sizes at a constant particle concentration of 0.0002%. Each individual exponent has an associated error of ± 0.025 before propagation.

Particle Size/nm	0	45	90	280	370	500
Particle Concentration/%	0	0.15	0.019	0.0006	0.003	0.0001
Exponents	0.523	0.430	0.405	0.325	0.362	0.306
	0.511		0.377	0.409	0.448	0.390
				0.463		0.430
						0.414
Standard Error	0.018	0.03	0.018	0.014	0.018	0.013

TABLE 4. Exponent values, each from a different defect pair, for varying nanoparticle sizes at a constant mass concentration of 0.1%. Each individual exponent has an associated error of ± 0.025 before propagation.

E Code: scaling law exponent graphs for varying nanoparticle concentration and constant nanoparticle size

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit

# Plot scaling exponent data and associated errors
x=[0,0.01,0.1,0.2,0.4]
y=[0.517,0.504,0.421,0.400,0.402]

yerr=[0.018,0.013,0.018,0.018,0.013]

plt.errorbar(x, y, yerr=yerr, fmt = "o", capsize = 4, markeredgecolor='red',markerfacecolor='red',  c='black')

# Axes labels
plt.xlabel('Nanoparticle Mass Concentration/%')
plt.ylabel('Scaling Exponent')
'''plt.title('Effect of Nanoparticle Concentration on Distance-Time \n
              Scaling Laws for Topological Defects in Solutions of Liquid Crystals')'''


# Taking exponential fit of the datapoints using SciPy
def exponential_decay(x,j,k,l):
    y = j*((1-k)**x)+l
    return y

fit_e = curve_fit(exponential_decay, xdata = x, ydata = y, p0 = [0.5, 0.5,0.5], sigma = yerr)
print('For an exponential fit y = j((1-k)^x)+l, j =', fit_e[0][0], ', k = ', fit_e[0][1], 'and l = ', fit_e[0][2])

# Use 3x3 covariant matrix to calculate error in constants
popt, pcov = fit_e
eerr = np.sqrt(np.diag(pcov))

print('The error in j is', eerr[0], ', the error in k is', eerr[1], ', and the error in l is', eerr[2], '.' )

# Create 1000 new datapoints in x across range of mass concentrations from 0-0.4%
x_new = np.linspace(0, 0.4, 1000)

# Plot exponential fit
yfit_e=[]
for coord in x_new:
   exponential = fit_e[0][0]*((1-fit_e[0][1])**coord)+fit_e[0][2]
   yfit_e.append(exponential)

plt.plot(x_new,yfit_e,color='blue')

plt.show()
```

FIG. 21. The code pictured was used to plot scaling exponents and associated errors at various mass concentrations for a given nanoparticle size of 280nm. The parameters of mass concentration and particle concentration can both be used to plot scaling exponents in this case as they both increase as more nanoparticles of constant size are added in solution. The code then takes an exponential fit of datapoints and prints the resultant constants, as well as plotting the fit in the same graph as the datapoints.

F Code: scaling law exponent graphs for varying nanoparticle size and constant nanoparticle concentration

```python
# Import libraries
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit

# Plot scaling exponent data and associated errors
x=[0,45,90,280,370,500]
y=[0.517,0.444,0.403,0.40,0.399,0.399]

yerr=[0.018,0.013,0.013,0.03,0.010,0.013]

plt.errorbar(x, y, yerr=yerr, fmt = "o", capsize = 4, markeredgecolor='red',markerfacecolor='red',  c='black')

# Axes labels
plt.xlabel('Nanoparticle Size/nm')
plt.ylabel('Scaling Exponent')
'''plt.title('Effect of Nanoparticle Size on Distance-Time Scaling \n
            Laws for Topological Defects in Solutions of Liquid Crystals')'''

# Taking exponential fit of the datapoints using SciPy
def exponential_decay(x,j,k,l):
    y = j*((1-k)**x)+l
    return y

fit_e = curve_fit(exponential_decay, xdata = x, ydata = y, p0 = [0.01, 0.3,0.35], sigma = yerr, bounds = [[0,0,0],[10,10,10]])
print('For an exponential fit y = j((1-k)^x)+l, j =', fit_e[0][0], ', k = ', fit_e[0][1], 'and l = ', fit_e[0][2])

# Use 3x3 covariant matrix to calculate error in constants
popt, pcov = fit_e
eerr = np.sqrt(np.diag(pcov))

print('The error in j is', eerr[0], ', the error in k is', eerr[1], ', and the error in l is', eerr[2], '.' )

# Create 1000 new datapoints in x across range of particle sizes from 0-500nm
x_new = np.linspace(0, 500, 1000)

# Plot exponential fit
yfit_e=[]
for coord in x_new:
   exponential = fit_e[0][0]*((1-fit_e[0][1])**coord)+fit_e[0][2]
   yfit_e.append(exponential)

plt.plot(x_new,yfit_e,color='blue')
plt.show()

```

FIG. 22. The code pictured was used to plot scaling exponents and associated errors at various nanoparticle sizes for a given nanoparticle concentration of 0.0002%. In this case, mass concentration increases as particle size increases, as the particle number is kept the same. The code then takes an exponential fit of datapoints and prints the resultant constants, as well as plotting the fit in the same graph as the datapoints.

G Code: scaling law exponent graphs for varying nanoparticle size and varying nanoparticle concentration

```
# Import libraries
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit

# Exponential prediction from varying mass concentration - prediction of 280nm, mass concentration 0.1%
s=280
c=0.1
vbothconc=0.121*((1-0.99999996)**c)+0.400
err_vbothconc = np.sqrt(0.0016*((4*0.00000001)**(2*c))+(9.4*0.00000000000000001*(c**2)*((4*0.00000001)**((2*c)-2)))+0.000009)
print(err_vbothconc)

# Exponential prediction from varying partice size - prediction of mass concentration 0.081%, 370nm
c=0.081
s=370
vsize=0.121*((1-0.024)**370)+0.398
err_vsize = np.sqrt((6.4*0.00001)*((0.976)**(2*s))+(1.32*0.0000001*(s**2)*((0.976)**((2*s)-2)))+0.000009)
print(err_vsize)

# Plot predictions
plt.errorbar(280,vbothconc,yerr=err_vbothconc,fmt = "o", capsize = 4, markeredgecolor='purple',markerfacecolor='purple',  c='purple')
plt.show()
plt.errorbar(370,vsize,yerr=err_vsize,fmt = "o", capsize = 4, markeredgecolor='green',markerfacecolor='green',  c='green')

#new data - plot scaling exponent data and associated errors
#use particle size on x axis
x=[0,45,90,280,370,500]
y=[0.517,0.43,0.391,0.399,0.405,0.385]

yerr=[0.018,0.03,0.018,0.014,0.018,0.013]

plt.errorbar(x, y, yerr=yerr, fmt = "o", capsize = 4, markeredgecolor='red',markerfacecolor='red',  c='black')

# Axes labels
plt.xlabel('Nanoparticle Size/nm')
plt.ylabel('Scaling Exponent')
'''plt.title('Effect of Nanoparticle Concentration and Size on Distance-Time \n
            Scaling Laws for Topological Defects in Solutions of Liquid Crystals')'''

# Taking exponential fit of the datapoints using SciPy
def exponential_decay(x,j,k,l):
    y = j*((1-k)**x)+l
    return y

fit_e = curve_fit(exponential_decay, xdata = x, ydata = y, p0 = [0.01, 0.3,0.35], sigma = yerr, bounds = [[0,0,0],[10,10,10]])
print('For an exponential fit y = j((1-k)^x)+l, j =', fit_e[0][0], ', k = ', fit_e[0][1], 'and l = ', fit_e[0][2])

# Use 3x3 covariant matrix to calculate error in constants
popt, pcov = fit_e
eerr = np.sqrt(np.diag(pcov))

print('The error in j is', eerr[0], ', the error in k is', eerr[1], ', and the error in l is', eerr[2], '.' )

```

FIG. 23. Section I of code used to plot scaling exponents and associated errors at various nanoparticle sizes for a given mass concentration of 0.1%. Keeping mass concentration constant while changing nanoparticle size results in variation in nanoparticle concentration. The code then takes an exponential fit of datapoints and prints the resultant constants. Predictions were also made of datapoints on the basis of exponential fits for scaling laws at constant particle size and for constant particle concentration.

```
51
52  print('The error in j is', eerr[0], ', the error in k is', eerr[1], ', and the error in l is', eerr[2], '.' )
53
54  # Create 1000 new datapoints in x across range of particle sizes from 0-500nm
55  x_new = np.linspace(0, 500, 1000)
56
57  # Plot exponential fit
58  yfit_e=[]
59  for coord in x_new:
60     exponential = fit_e[0][0]*((1-fit_e[0][1])**coord)+fit_e[0][2]
61     yfit_e.append(exponential)
62
63  plt.plot(x_new,yfit_e,color='blue')
64
65  # Create labels for graphical data within legend
66  plt.legend(["Exponential Fit","Prediction:Constant Size","Prediction: Constant Particle Concentration"])
67  plt.show()
```

FIG. 24. Section II of code used to plot scaling exponents and associated errors at various nanoparticle sizes for a given mass concentration of 0.1%. The code plots the exponential fit onto the same graph with the experimental datapoints.

H Code for calculating nanoparticle concentration from mass concentration and nanoparticle size

```python
import numpy as np

pi=np.pi

particle_size = float(input("Enter the particle size in nanometers. \n"))
particle_size = particle_size*0.0000001
mass_concentration = float(input("Enter the mass concentration as a percentage. \n"))
mass_concentration=mass_concentration/100
# Density of barium titanate nanoparticles is 6.02 grams per centimetre cubed
mass_nanoparticle = (4/3)*pi*((particle_size/2)**3)*6.02

# Liquid crystal MBBA has molecular mass of 267.372 grams per mole
mass_MBBA = 267.372/(6.02*(10**23))

upper = mass_concentration/mass_nanoparticle
lower=(upper)+((1-mass_concentration)/mass_MBBA)

nanoparticle_concentration = upper/lower
nanoparticle_concentration = nanoparticle_concentration*(10**8)
print(nanoparticle_concentration)

```

FIG. 25. Nanoparticles were approximated as spheres of constant density. Therefore, this code was used to calculate nanoparticle concentration for a given mass concentration and size of nanoparticles.